LAPORAN PRAKTIKUM PEMROGRAMAN BASIS DATA
WEEK 5: FUNGSI PADA BASIS DATA (POSTGRESQL)

LAPORAN PRAKTIKUM



Oleh

MUHROMIN

NIM 25051214046

UNIVERSITAS NEGERI SURABAYA

FAKULTAS TEKNIK

PROGRAM STUDI S1 SISTEM INFORMASI

2026

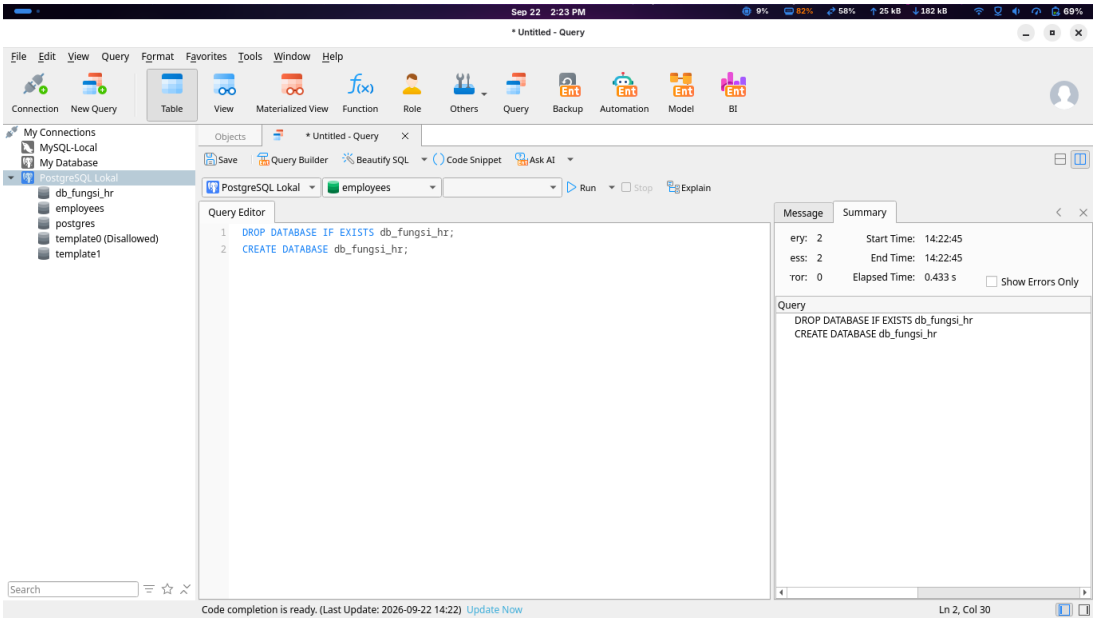
Ringkasan Praktikum

Praktikum ini membahas pembuatan dan pemanfaatan fungsi (stored function) pada basis data, meliputi fungsi skalar murni, fungsi yang membaca tabel, penerapan dalam skenario nyata, pengujian, penelusuran galat, dan pengukuran kinerja.

Seluruh praktikum dijalankan pada PostgreSQL 18 dengan basis data kerja db_fungsi_hr. Soal aslinya ditulis untuk MySQL sehingga dilakukan penyesuaian sintaks. Data diambil dari dataset employees melalui foreign table, dengan rincian sebagai berikut

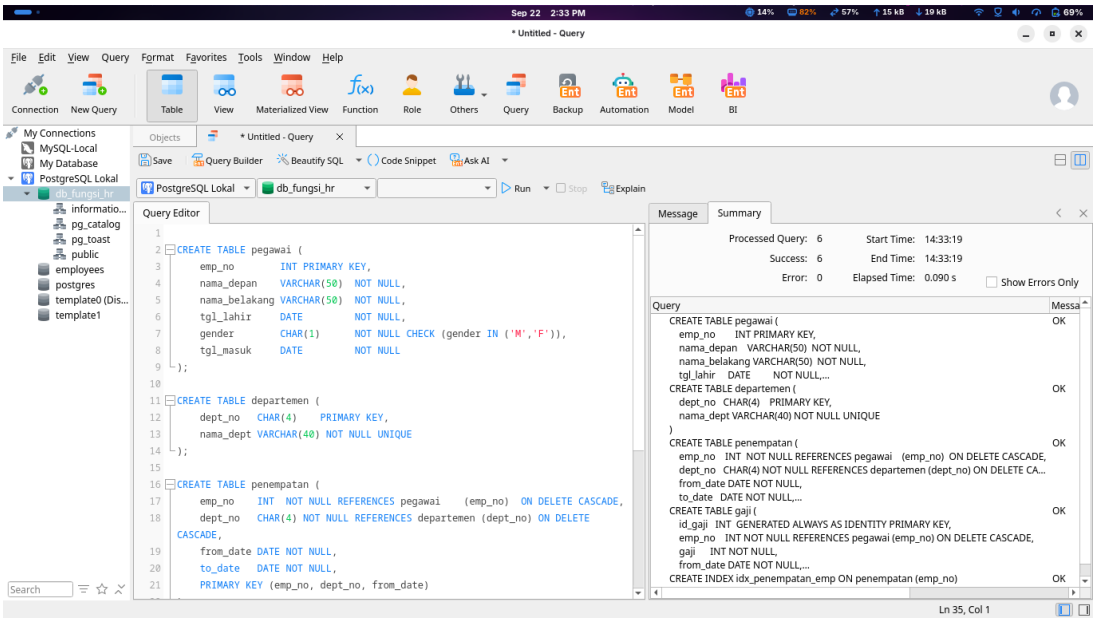
departemen	pegawai	penempatan	gaji	uji_fungsi
9	5.000	5.524	47.603	16

A. Persiapan



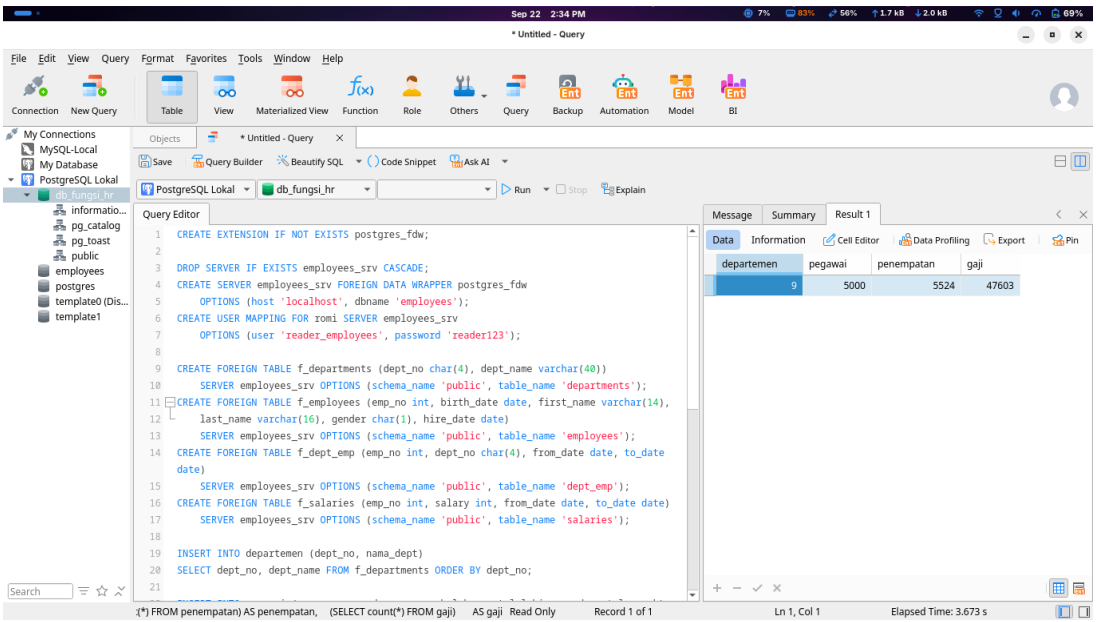
Gambar 1: Database db_fungsi_hr berhasil dibuat.

Dibuat database kerja sebagai tempat seluruh objek praktikum.



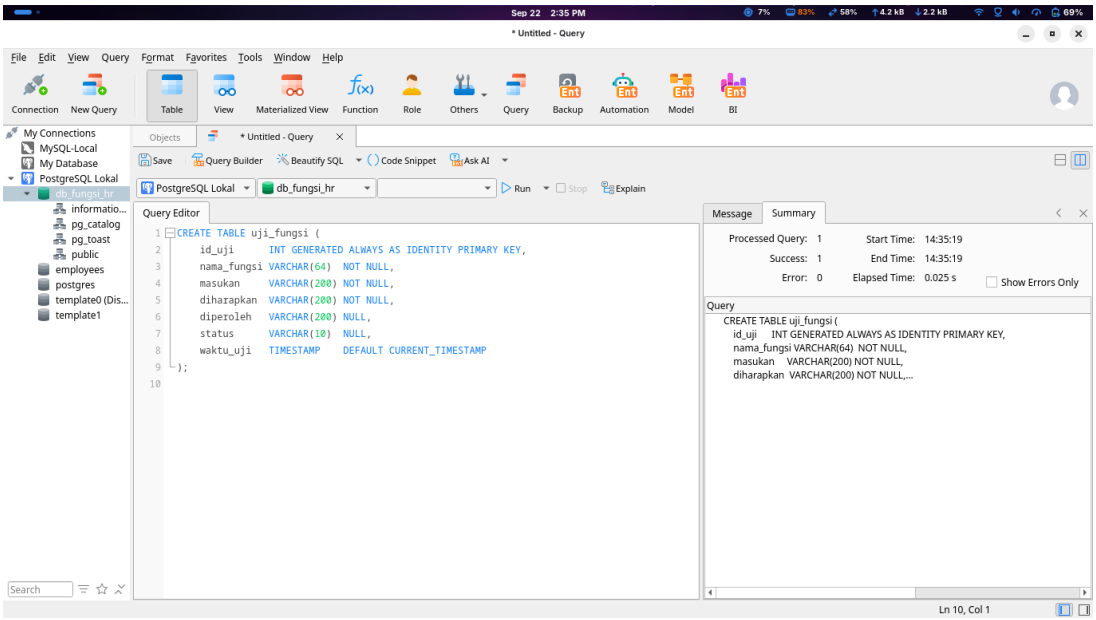
Gambar 2: Keempat tabel operasional berhasil dibuat.

Empat tabel operasional dibuat, yaitu pegawai, departemen, penempatan, dan gaji, lengkap dengan kunci utama, kunci asing, dan indeks.



Gambar 3: Jumlah baris keempat tabel setelah pengisian data.

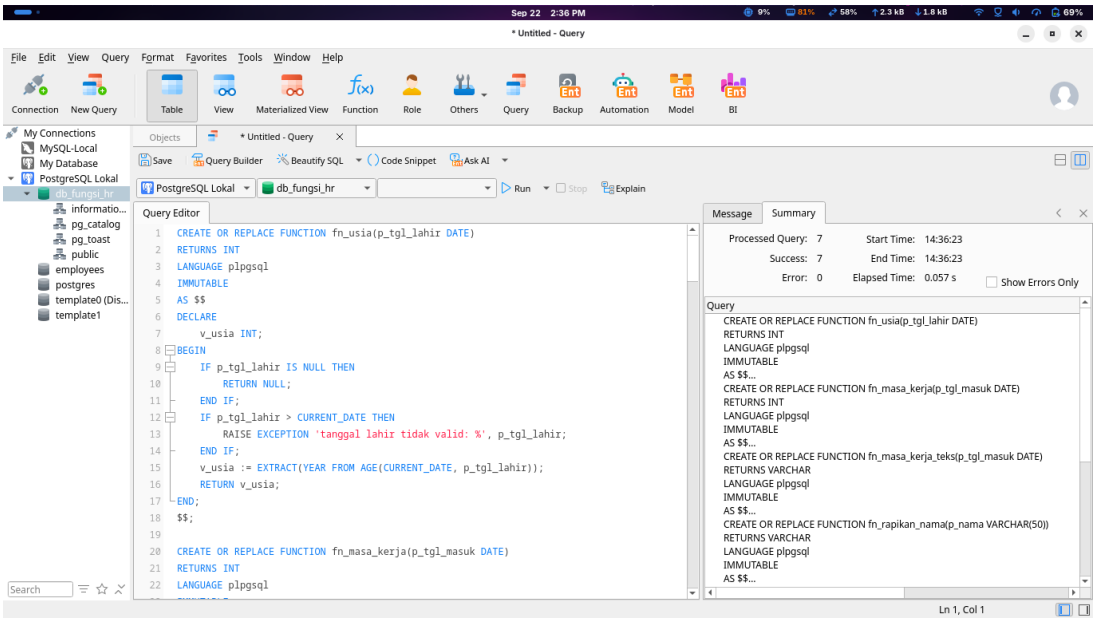
Data diisi dari database employees melalui foreign table (postgres_fdw).



Gambar 4: Tabel uji_fungsi berhasil dibuat.

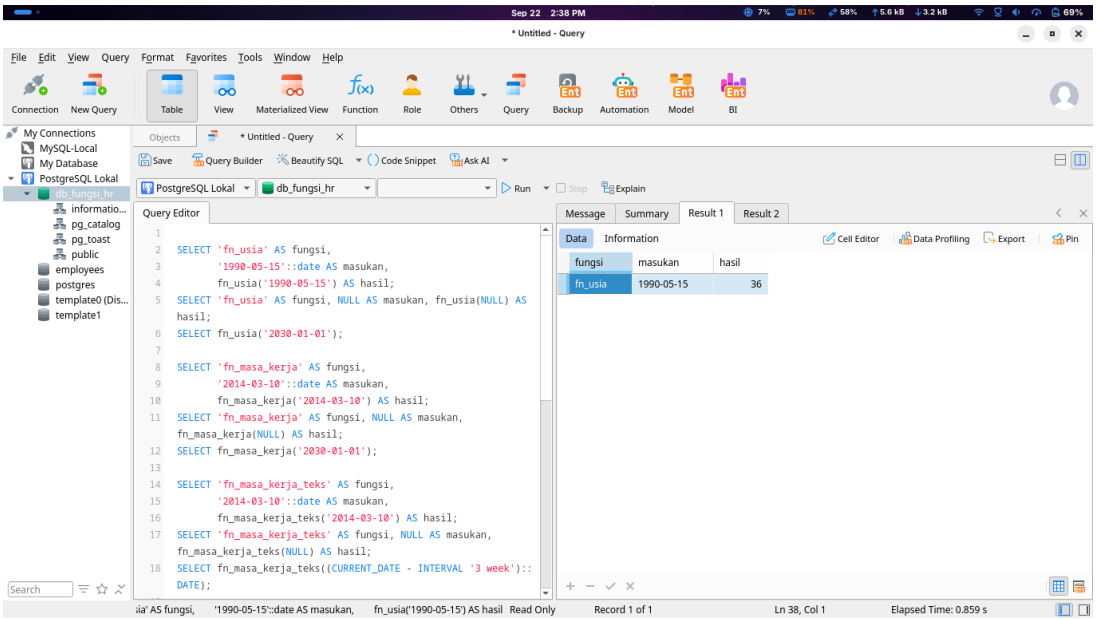
Tabel pendukung untuk mencatat hasil pengujian terpusat.

B. Fungsi Skalar Murni



Gambar 5: Enam fungsi skalar murni berhasil dibuat.

Enam fungsi dibuat, yaitu fn_usia, fn_masa_kerja, fn_masa_kerja_teks, fn_rapikan_nama, fn_format_rupiah, dan fn_kategori_gaji. Fungsi ini tidak menyentuh tabel, dan hasilnya dihitung dari parameter.

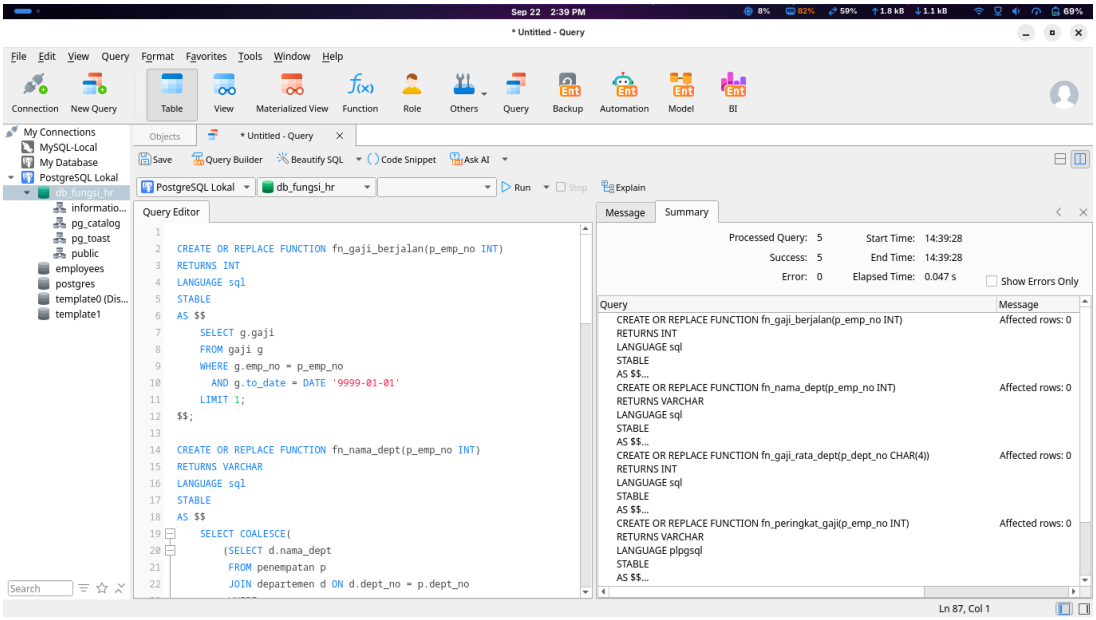


Gambar 6: Hasil pengujian fungsi Bagian B (termasuk penolakan input tak valid).

Setiap fungsi diuji dengan nilai normal, NULL, dan nilai tidak wajar (tanggal masa depan, gaji negatif). Fungsi mengembalikan NULL untuk input NULL dan menolak input tak valid dengan RAISE EXCEPTION.

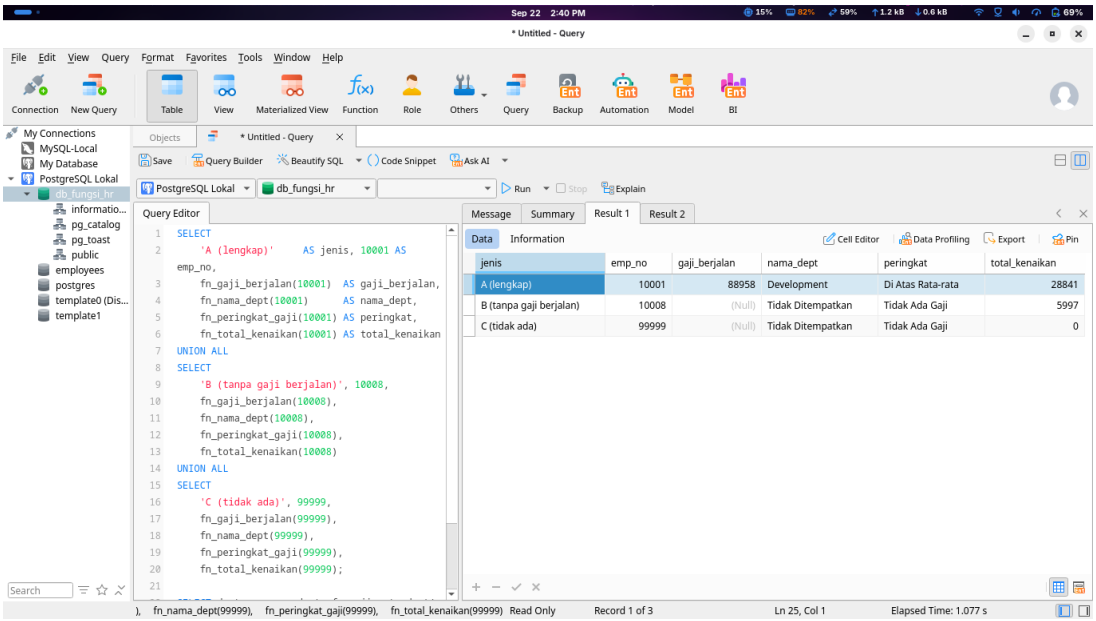
Analisis: fn_usia bergantung pada CURRENT_DATE sehingga hasilnya berubah seiring waktu. Bila dipaksa IMMUTABLE, mesin basis data boleh meng-cache hasilnya sehingga pada replikasi node akan menampilkan usia yang usang, dan hasil query yang disimpan sementara (view materialized) akan membeku. Maka fungsi ini tidak boleh dideklarasikan IMMUTABLE.

C. Fungsi yang Membaca Tabel



Gambar 7: Lima fungsi pembaca tabel berhasil dibuat.

Lima fungsi STABLE yang membaca tabel, yaitu fn_gaji_berjalan, fn_nama_dept, fn_gaji_rata_dept, fn_peringkat_gaji, serta fn_total_naik.



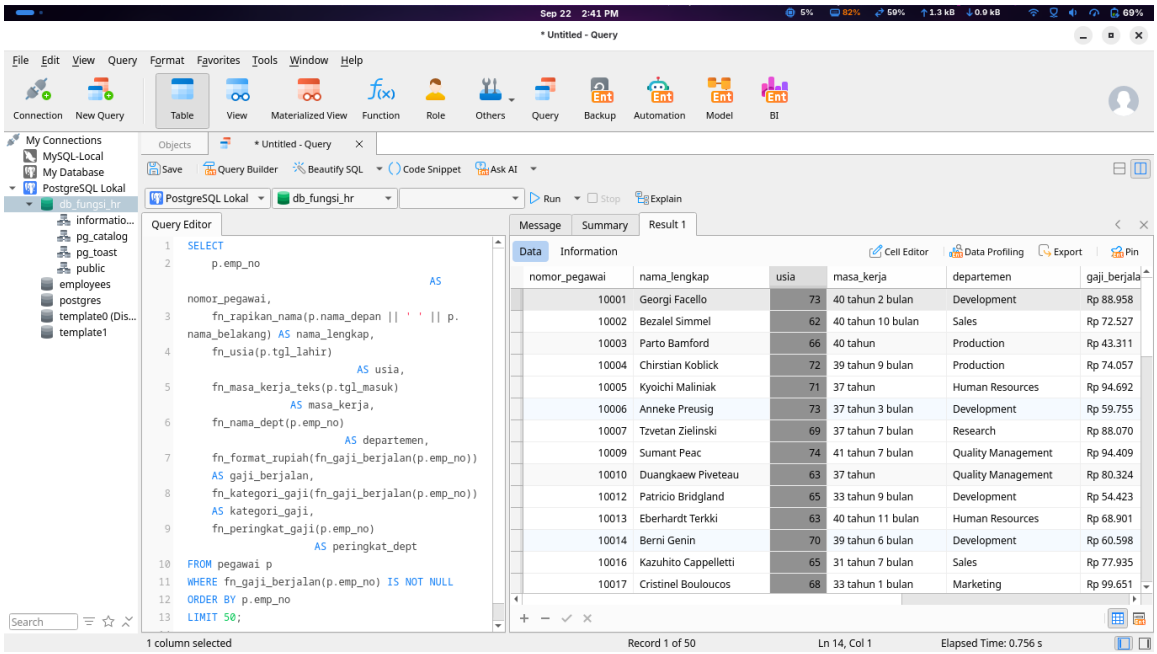
Gambar 8: Pengujian fungsi Bagian C dengan tiga jenis masukan.

Fungsi diuji dengan tiga jenis emp_no, yaitu (A) data lengkap, (B) ada pegawai tetapi tanpa gaji berjalan, dan (C) pegawai tidak ada.

Analisis: Penyusunan fungsi secara berlapis memberi reuse dan konsistensi (logika ditulis sekali, dipakai di mana saja) serta keterbacaan dan pemeliharaan (query laporan pendek, aturan cukup diubah di satu tempat). Risikonya, kinerja dan ketergantungan berantai, yaitu tiap lapisan menambah panggilan sehingga memperlambat, dan perubahan/penghapusan fungsi dasar berdampak kaskade ke semua pemanggil.

D. Penerapan dalam Skenario Nyata

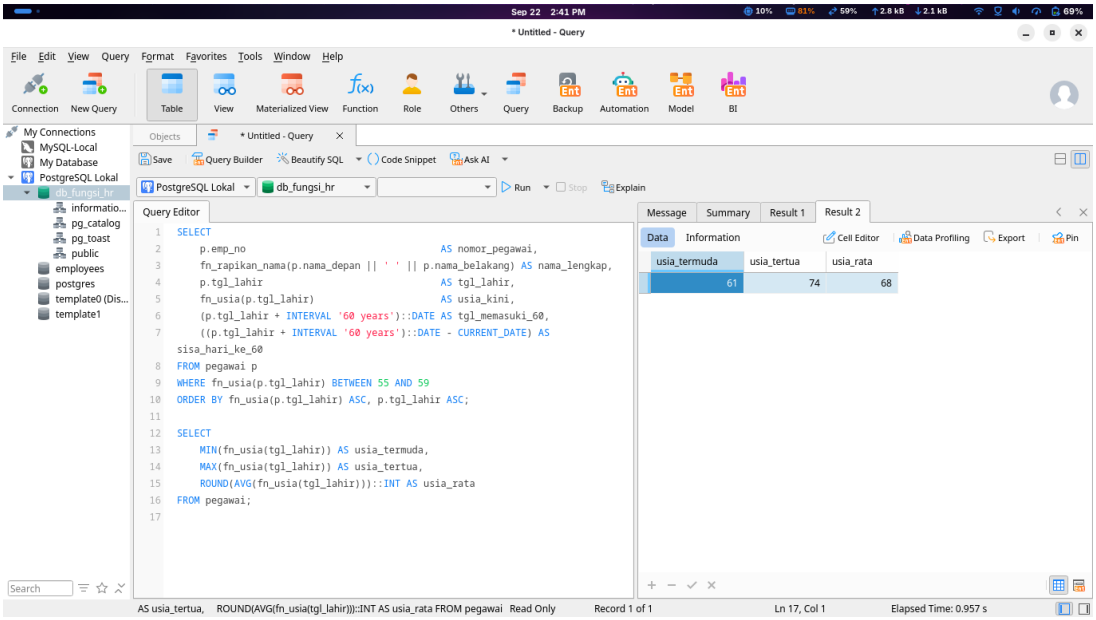
D1. Laporan Profil Pegawai



Gambar 9: Laporan profil pegawai (D1).

Laporan 50 baris, dan seluruh kolom turunan berasal dari pemanggilan fungsi.

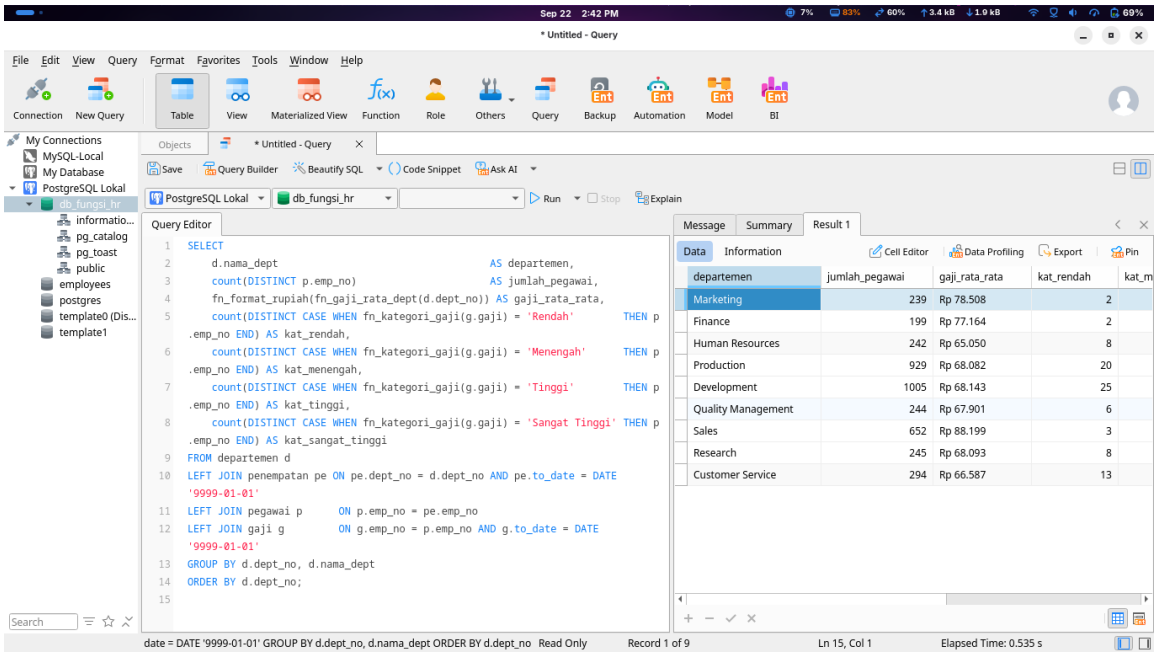
D2. Proyeksi Pensiun



Gambar 10: Proyeksi pensiun (D2), 0 baris karena seluruh pegawai sudah lebih dari 60 tahun

Query memakai fn_usia di WHERE dan ORDER BY. Pada dataset ini seluruh pegawai sudah berusia di atas 60 tahun sehingga hasil 0 baris, dan temuan ini dilaporkan apa adanya.

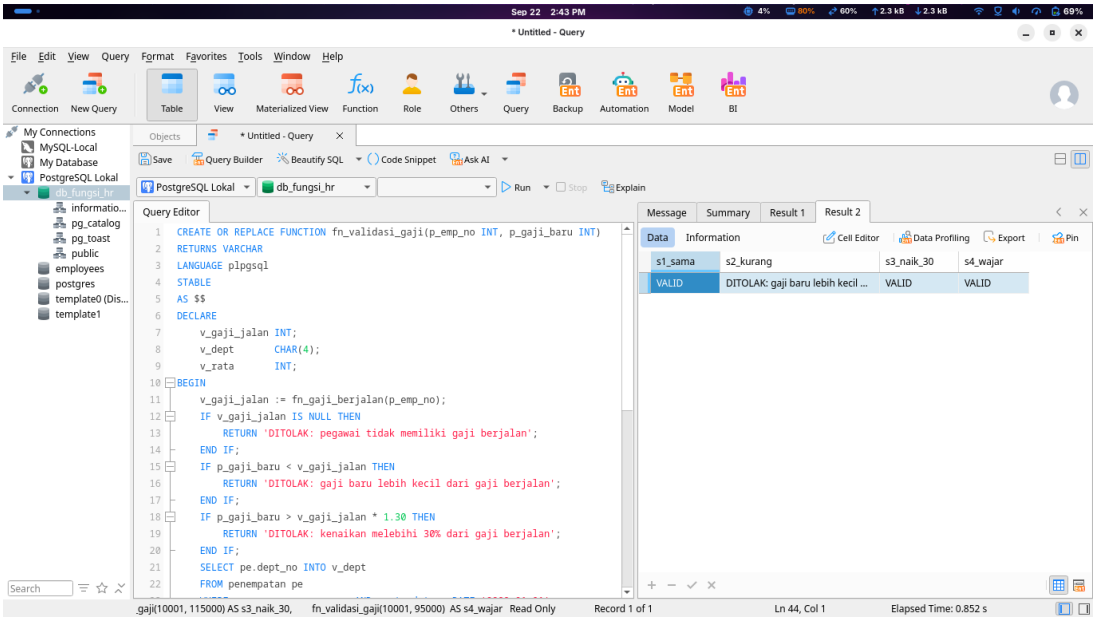
D3. Rekap per Departemen



Gambar 11: Rekapitulasi per departemen (D3).

Rekapitulasi per departemen meliputi jumlah pegawai, gaji rata-rata (rupiah), dan sebaran kategori gaji.

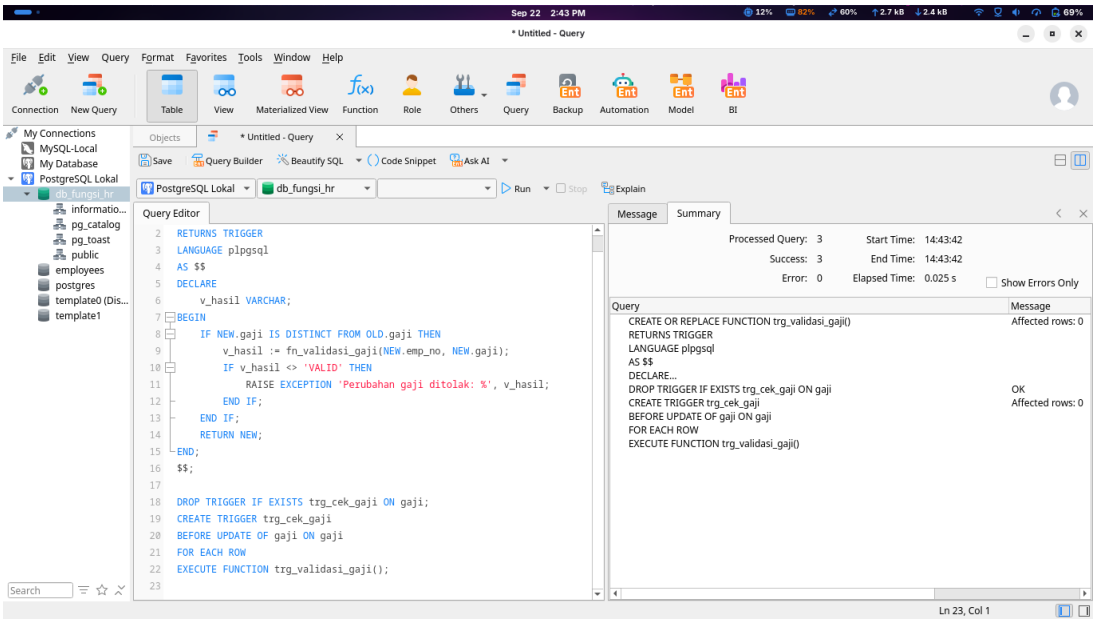
D4. Validasi Gaji



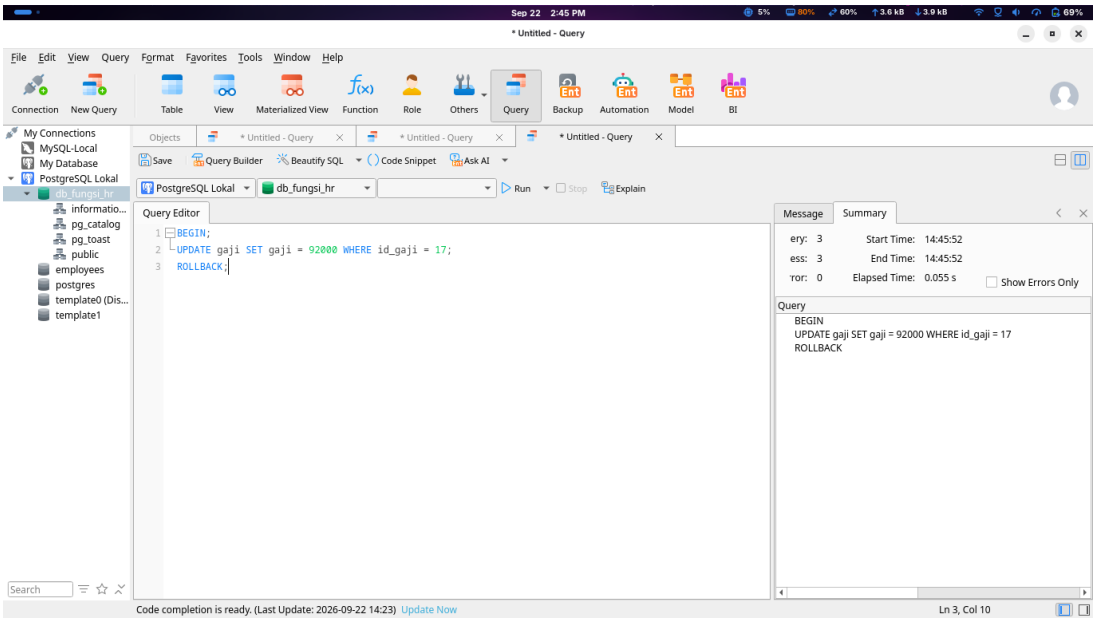
Gambar 12: Hasil validasi gaji beberapa skenario (D4).

fn_validasi_gaji mengembalikan VALID atau pesan penolakan (gaji baru kurang dari berjalan, kenaikan lebih dari 30%, atau lebih dari 2 kali rata-rata departemen).

D5. Trigger Validasi Gaji



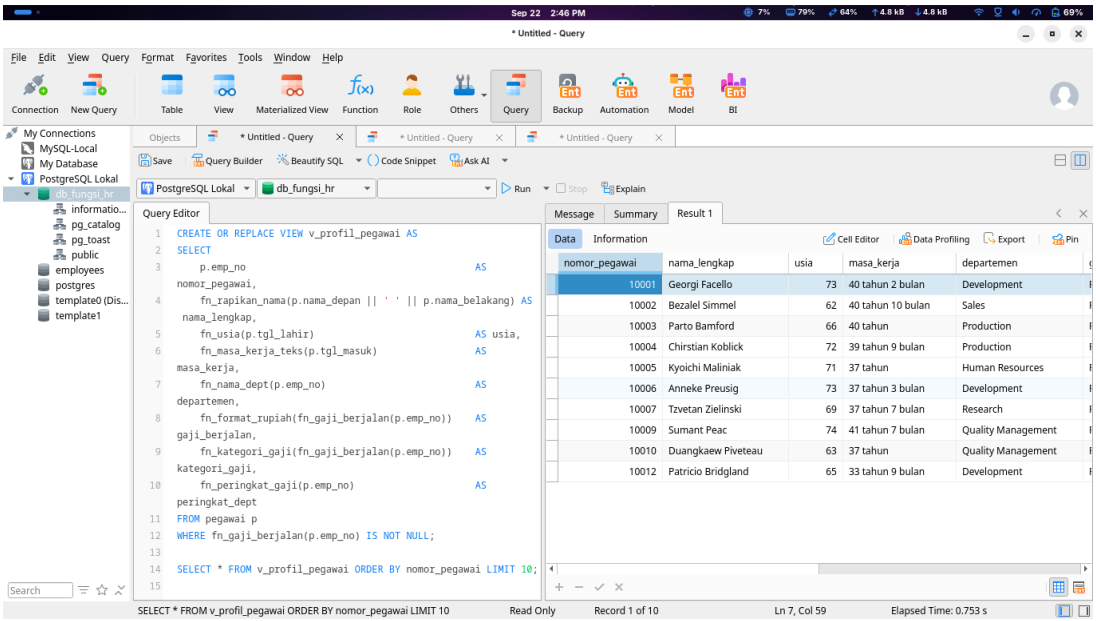
Gambar 13: Trigger trg_cek_gaji berhasil dibuat.



Gambar 14: Trigger menolak UPDATE tidak valid (ERROR) dan mengizinkan yang valid.

Trigger BEFORE UPDATE menolak perubahan bila hasil validasi bukan VALID (menggunakan RAISE EXCEPTION yang merupakan padanan SIGNAL SQLSTATE 45000).

D6. View



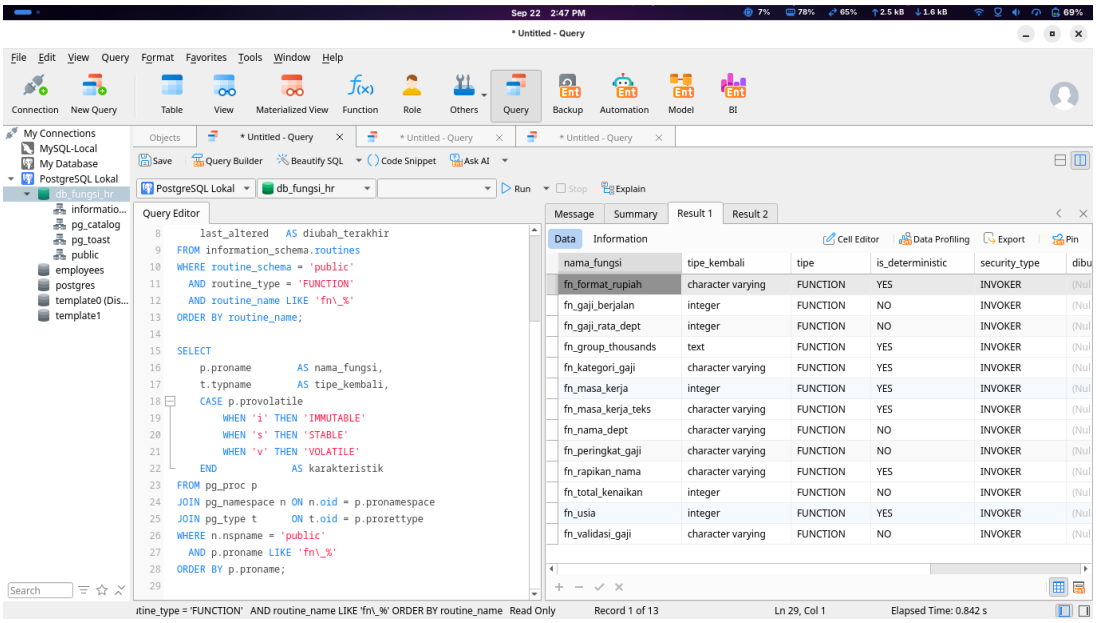
Gambar 15: Pemanggilan view v_profil_pegawai (D6).

View membungkus query D1 sehingga cukup dipanggil dengan `SELECT * FROM v_profil_pegawai`.
Analisis: Fungsi terbaik untuk logika kompleks/bersyarat dengan parameter. View terbaik untuk presentasi berulang yang selalu live. Generated column terbaik untuk nilai turunan stabil

yang sering difilter/disortir karena tersimpan fisik dan bisa diindeks.

E. Pengujian, Penelusuran, dan Pengelolaan

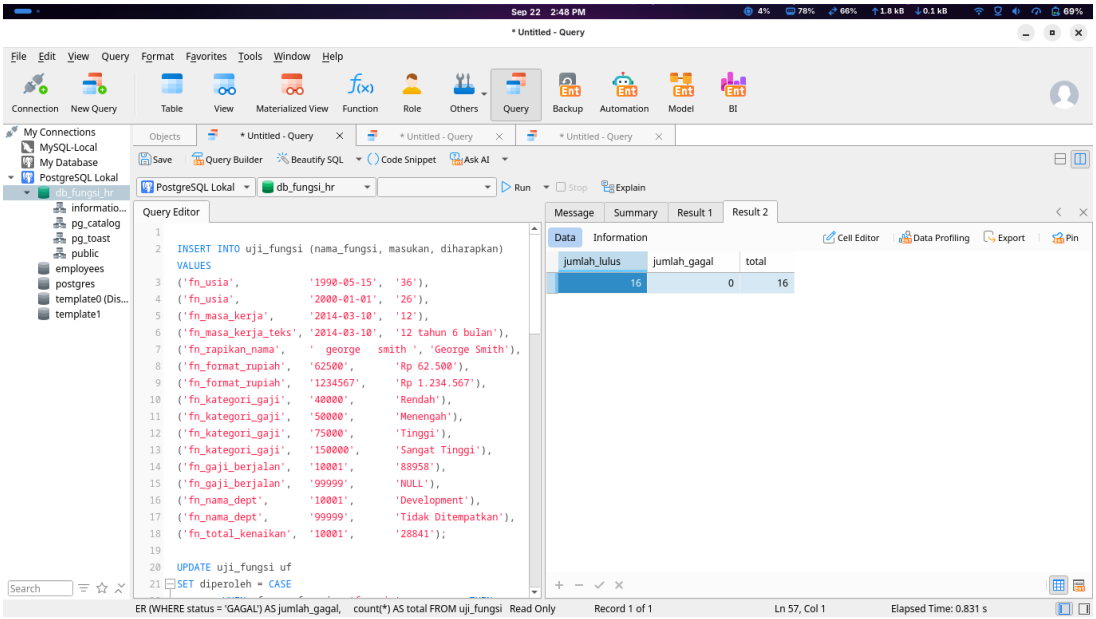
E1. Inventaris Fungsi



Gambar 16: Inventaris fungsi (E1).

Daftar seluruh fungsi buatan beserta tipe, determinisme, dan waktu pembuatan diambil dari information_schema.ROUTINES dan pg_proc.

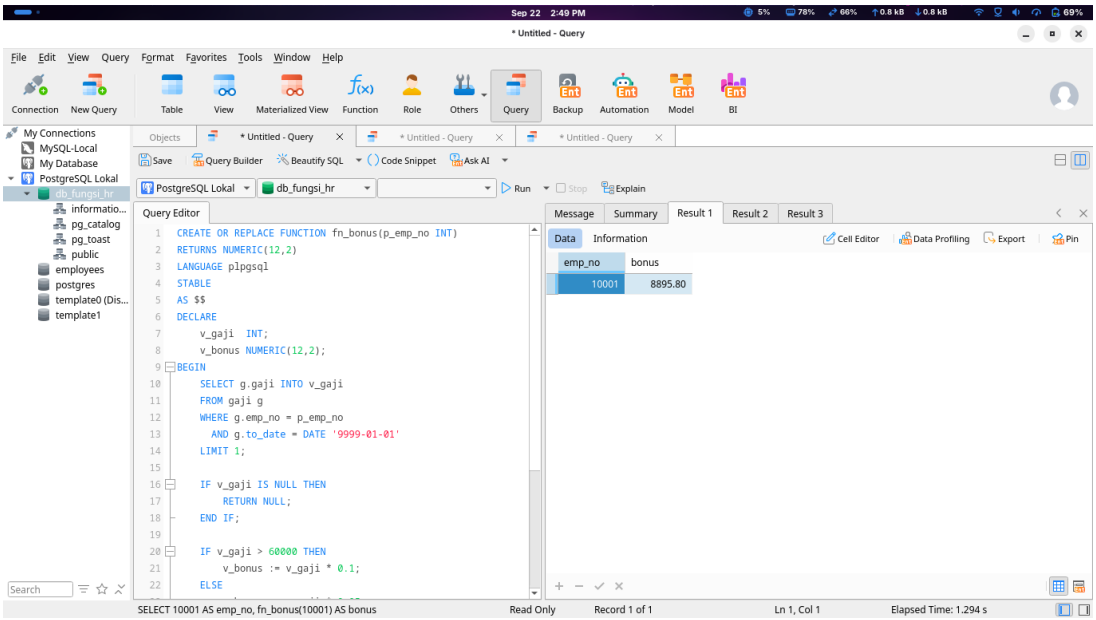
E2. Pengujian Terpusat



Gambar 17: Rekapitulasi hasil pengujian, 16 lulus dan 0 gagal (E2)

Enam belas kasus uji dicatat di tabel uji_fungsi, dan seluruhnya LULUS.

E3. Penelusuran Galat



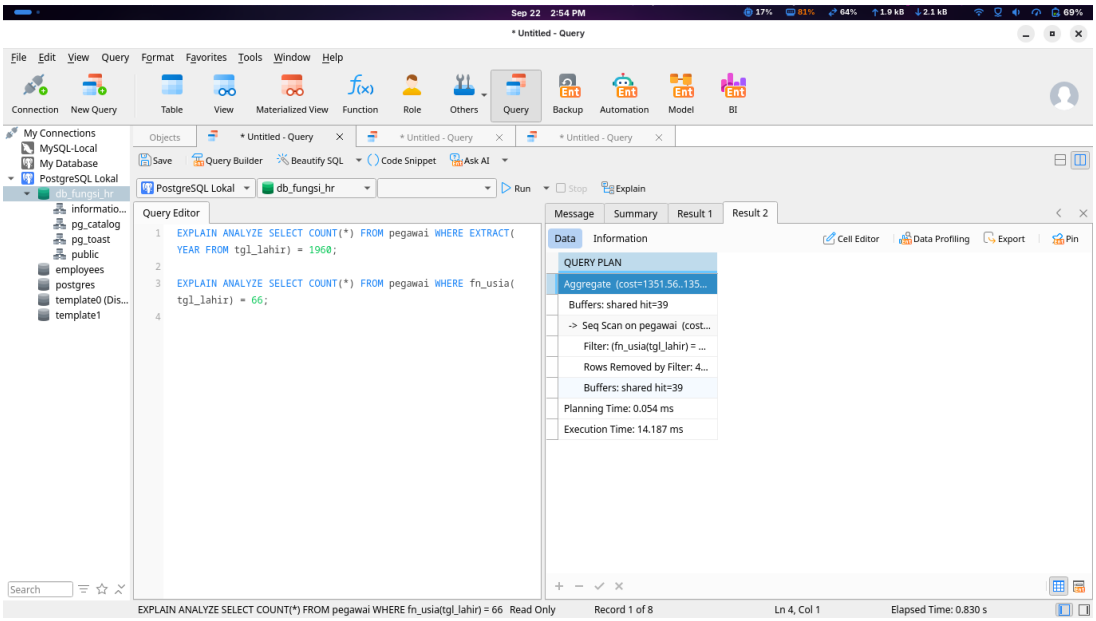
Gambar 18: Hasil fn_bonus versi perbaikan (E3).

Skrip fn_bonus yang diberikan mengandung enam kesalahan. Pertama, v_bonus tidak dideklarasikan. Kedua, tidak ada RETURN. Ketiga, SELECT...INTO tanpa LIMIT (error bila

lebih dari satu baris). Keempat, tidak menangani NULL pada v_gaji. Kelima, pengali desimal pada variabel INT memotong presisi. Keenam, penggunaan DELIMITER yang tidak dikenali PostgreSQL. Versi perbaikan menghasilkan nilai bonus yang benar.

Analisis: Dua teknik penelusuran logika fungsi, yaitu RAISE NOTICE/INFO/DEBUG yang ringan namun keluaran masuk ke log dan bising bila berlebihan. Kedua, menyimpan jejak ke tabel audit yang lebih terstruktur namun menambah I/O dan bersifat efek samping (tidak cocok untuk fungsi IMMUTABLE/STABLE).

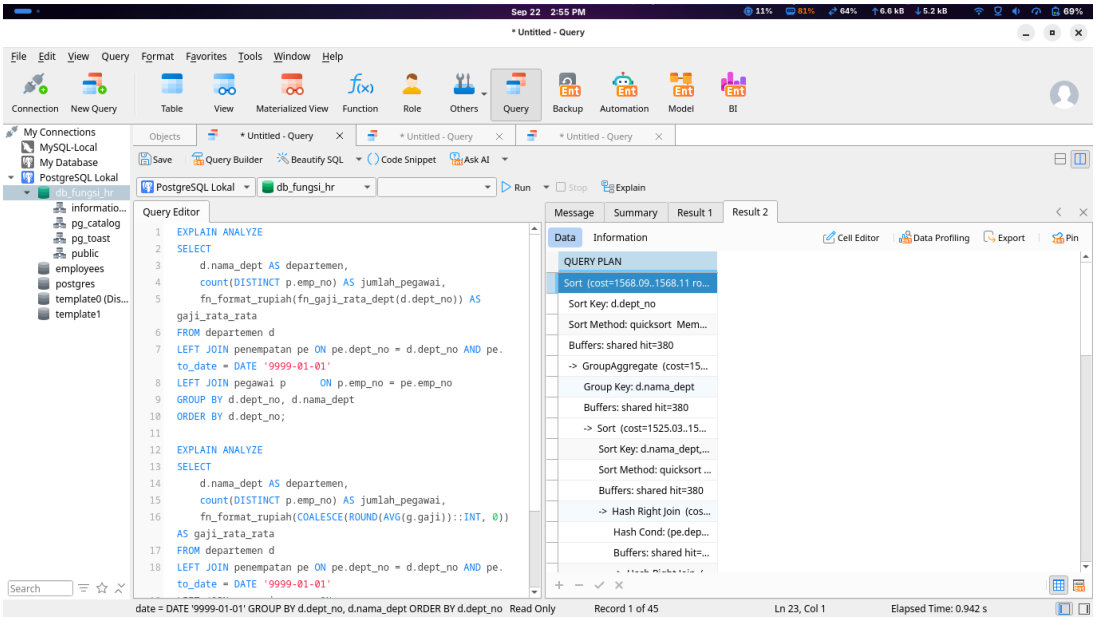
E4. Pengukuran Kinerja, Kolom vs Fungsi



Gambar 19: Benchmark E4, kolom polos vs dibungkus fungsi

EXPLAIN ANALYZE membandingkan filter pada kolom polos versus kolom yang dibungkus fn_usia.

E5. Pengukuran Kinerja, Fungsi vs JOIN



Gambar 20: Benchmark E5, fungsi vs JOIN+GROUP BY

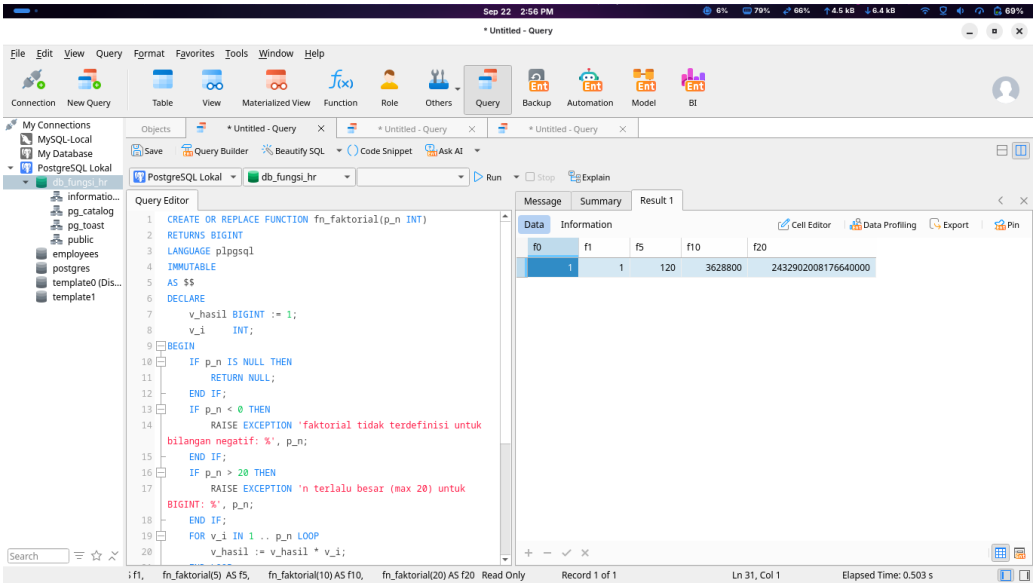
EXPLAIN ANALYZE membandingkan rekap departemen yang memakai fn_gaji_rata_dept versus versi JOIN + GROUP BY.

Analisis: predikat yang membungkus kolom dengan fungsi membuat indeks pada kolom mentah tidak terpakai (terjadi Seq Scan). Untuk tetap memakai indeks diperlukan expression index pada ekspresi tersebut. Perbandingan waktu eksekusi sebagai berikut

Tabel 1: Perbandingan waktu eksekusi (E4 & E5).

Ukuran	Langsung (kolom/JOIN)	Ber-fungsi
E4 (filter 5.000 baris)	∼2 ms	∼10 ms
E5 (rekap departemen)	∼24 ms	∼78 ms

E7. Faktorial

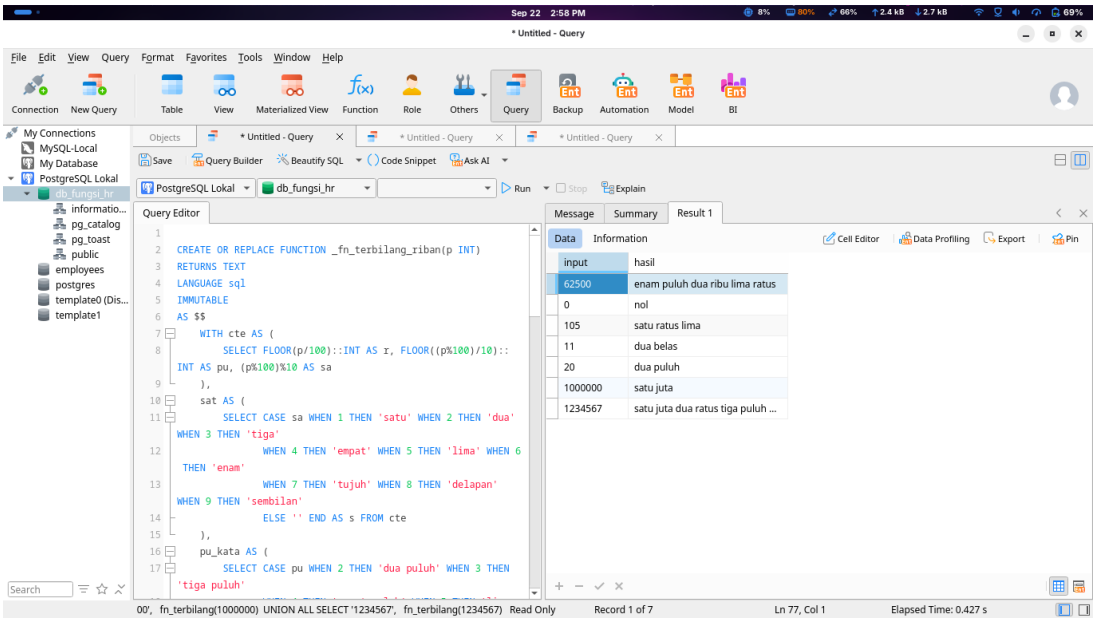


Gambar 21: Hasil fn_faktorial (E7).

PostgreSQL (seperti MySQL) tidak mendukung rekursi fungsi, sehingga fn_faktorial diimplementasikan dengan loop iteratif FOR.

Tugas Tambahan

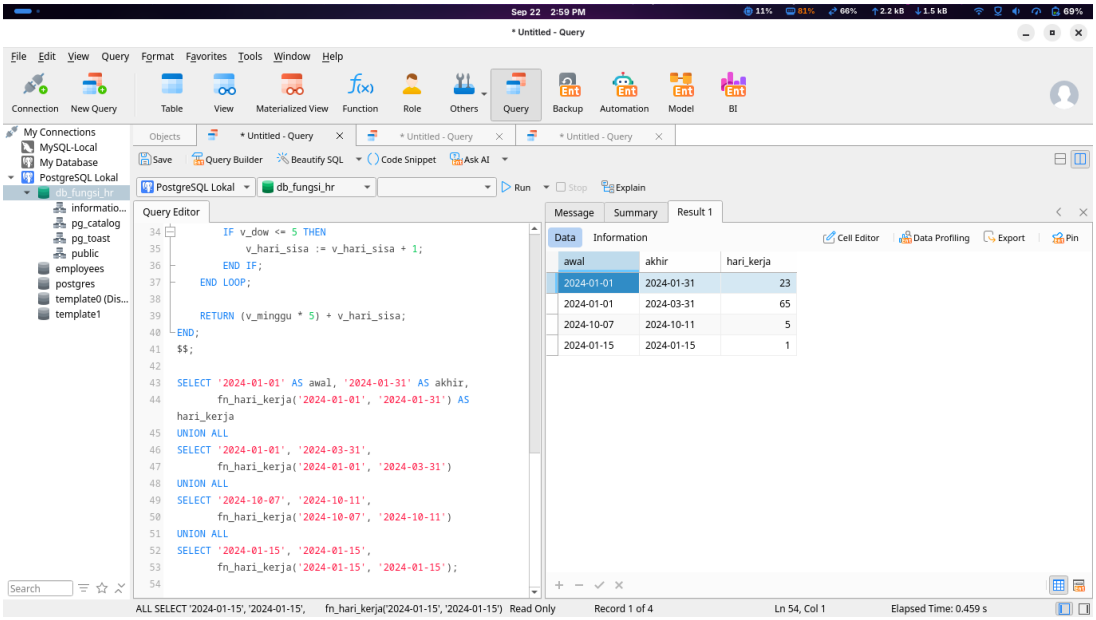
G1. Terbilang



Gambar 22: Hasil fn_terbilang (G1).

fn_terbilang mengubah bilangan menjadi kata bahasa Indonesia sampai jutaan, misalnya 62500 menjadi “enam puluh dua ribu lima ratus”.

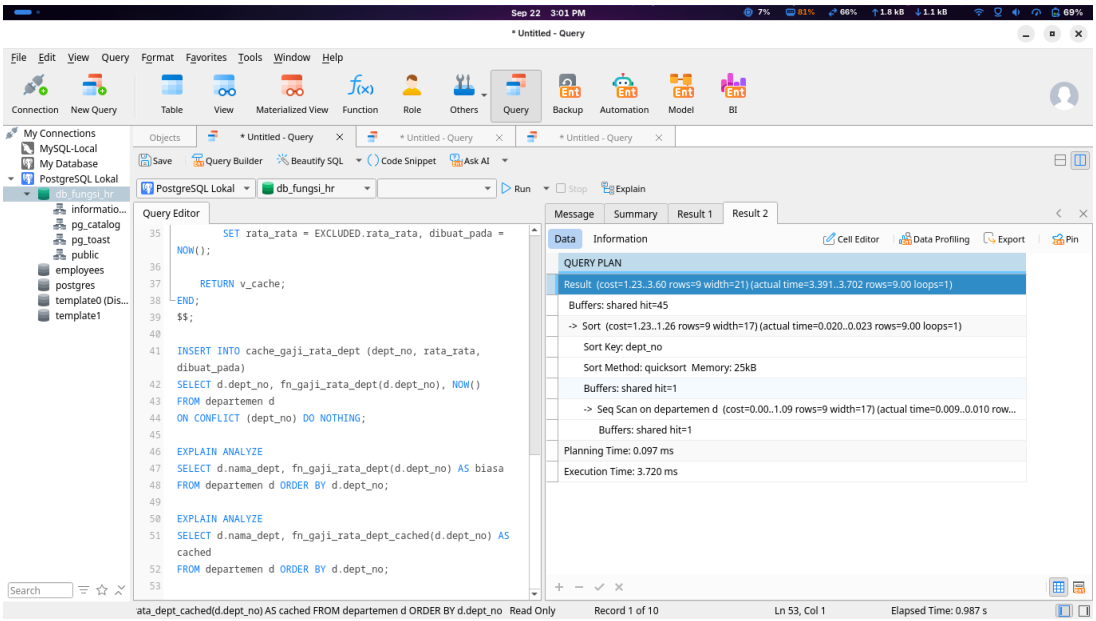
G2. Hari Kerja



Gambar 23: Hasil fn_hari_kerja (G2).

fn_hari_kerja menghitung hari kerja dari Senin sampai Jumat antara dua tanggal dengan rumus matematis (minggu penuh $\times$ 5 + sisa), tanpa perulangan baris-per-baris.

G3. Caching



Gambar 24: Benchmark caching (G3), versi biasa vs versi cached

Mekanisme caching fn_gaji_rata_dept memakai tabel cache_gaji_rata_dept dengan masa kedaluwarsa 1 jam.

Analisis: caching mempercepat query secara signifikan (versi biasa $\sim$58 ms menjadi versi cached $\sim$1 ms pada data yang sama). Risikonya: bila data gaji berubah dalam jendela 1 jam, cache menyajikan nilai yang usang sampai kedaluwarsa. Untuk data yang berubah sering, kedaluwarsa perlu diperpendek atau cache di-invalidasi secara eksplisit.

Simpulan

Fungsi basis data sangat berguna untuk menyusun logika bisnis yang berulang menjadi pendek, seragam, dan mudah dibaca. Praktikum ini membuktikan:

1. Fungsi skalar murni (Bagian B) dan fungsi pembaca tabel (Bagian C) dapat dibuat, diuji, dan dirangkai secara berlapis dengan aman.

2. Penerapan fungsi dalam skenario nyata (Bagian D) meliputi profil, validasi gaji, trigger, dan view. Semua berjalan sesuai ketentuan, dan trigger menolak perubahan yang tidak valid.

3. Pengujian terpusat (E2) menunjukkan 16 dari 16 kasus LULUS.

4. Fungsi yang membungkus kolom pada data besar menurunkan kinerja karena indeks tidak terpakai (Bagian E4 dan E5). Caching (G3) terbukti memberi percepatan signifikan.

Tiga pedoman, yaitu gunakan fungsi untuk logika kompleks yang berulang. Kedua, hindari membungkus kolom dengan fungsi pada predikat query besar (gunakan expression index bila perlu). Ketiga, gunakan caching atau generated column untuk agregat berulang yang mahal.